Jacobian Matrix and Jacobian Scale Factor

Use in Kalman Filters and Multivariable Calculus

CS3630 Intro to Perception and Robotics
Georgia Institute of Technology

Created by Mukilan Karthikeyan

Contents

1 The Jacobian Matrix vs The Jacobian Scale Factor

1.1 The Jacobian Matrix

Given a vector-valued function takes in n-inputs and has m-outputs

$$f : R^n \to R^m$$

the Jacobian Matrix is defined as the full matrix of all partial derivatives with dimension m x n. This tells how much each output changes with respect to each input.

$$\mathbf{J} = \frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_n} \\ \vdots & & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

1.2 The Jacobian Scale Factor

When the Jacobian matrix is square, $m = n$, we can take the determinant:

$$|\mathbf{J}| = |\det(\frac{\partial \mathbf{f}}{\partial \mathbf{x}})|$$

Which provides a scalar value or the Jacobian Scale Factor, which is the absolute value of the determinant of the Jacobian matrix (we only care about the squash or stretch without regard for the direction). This is the factor by which any given volume shrinks or stretches under a given transformation (the transformation in this case being the function f). This scalar is still a function that needs to be evaluated at specific values if we want a numerical value. This is why when we change variables in integrals, we must use the Jacobian Scale Factor $|\mathbf{J}|$:

$$\iint_R f(x, y)dxdy = \iint_S f(g(u, v), h(u, v)) \cdot |\det(J(u, v))|dudv$$

where we change our integration over x, y to u, v with $x = g(u, v)$ and $y = h(u, v)$. The Jacobian Matrix here is defined as:

$$\mathbf{J} = \begin{bmatrix} \frac{\partial x}{\partial u} & \frac{\partial x}{\partial v} \\ \frac{\partial y}{\partial u} & \frac{\partial y}{\partial v} \end{bmatrix}$$

For example, when you are converting from Cartesian coordinates to Polar coordinates:

$$x = r\cos(\theta), \quad y = r\sin(\theta)$$

$$\mathbf{J} = \begin{bmatrix} \cos\theta & -r\sin(\theta) \\ \sin\theta & r\cos(\theta) \end{bmatrix}, \quad |\mathbf{J}| = r$$

	Jacobian Matrix	**Jacobian Scale Factor**
Multi-variable calculus	Chain rule for vector functions	Change of variables in integrals
Kalman filters	Linearizing nonlinear functions	Rarely used directly
Optimization	Gradient computation, Gauss-Newton	Normalization in probability densities
Robotics	Relating joint velocities to end-effector velocities	Workspace volume analysis

2 The EKF Jacobian: Local Linearization

2.1 Why linearization is needed

The standard Kalman filter assumes everything is linear:

$$\mathbf{x}_k = F\mathbf{x}_{k-1} + \mathbf{w}, \quad \mathbf{z}_k = H\mathbf{x}_k + \mathbf{v}$$

Note

Here we are using F as the system dynamics matrix, in class and in the notes this is denoted with A.
While the Kalman Filter is an optimal estimator, the Extended Kalman Filter is only an optimal estimator under certain conditions. Thus the EKF is generally not considered an optimal estimator.

But real systems have nonlinear dynamics $f(\cdot)$ and nonlinear measurements $h(\cdot)$:

$$\mathbf{x}_k = f(\mathbf{x}_{k-1}) + \mathbf{w}, \quad \mathbf{z}_k = h(\mathbf{x}_k) + \mathbf{v}$$

You can't directly apply the Kalman equations to nonlinear functions because they rely on the propagation of Gaussian distributions representing uncertainty; Gaussian distributions under nonlinear transformations are no longer Gaussian. The EKF sovles this problem by approximate the nonlinear functions as linear around the current best estimate, using the Jacobian.This is a first order Taylor expansion.

Note

If you are familiar with the Euler method for solving Ordinary Differential Equations (ODEs) this is similar in concept. Both are First Order Taylor Approximations used to estimate different things. Refer to the "EKF Connection to Euler Method for ODE" for more information

2.2 Example working with notation

Lets work through a example to familiarize ourselves with how these ideas show up in notation. Lets take an example where we have a robot exploring a location. It has access to a map of the environment and a sensor reading of its distance to a beacon (similar structure to using GPS where the satellites are the beacons).

$$\mathbf{x} = \begin{bmatrix} p_x \\ p_y \\ \dot{p}_x \\ \dot{p}_y \end{bmatrix}, \quad h(\mathbf{x}) = \sqrt{p_x^2 + p_y^2}$$

$$h(x) \approx h(\bar{x}_k) + \left.\frac{\partial h}{\partial \mathbf{x}}\right|_{\bar{x}_k} (x - \bar{x}_k)$$

With EKF we define the Measurement Jacobian H evaluated at $\bar{x}$ for our Measurement update step:

$$H_k \quad = \quad \left.\frac{\partial h}{\partial \mathbf{x}}\right|_{\bar{x}_k} \quad = \quad \begin{bmatrix} \frac{p_x}{\sqrt{p_x^2+p_y^2}} & \frac{p_y}{\sqrt{p_x^2+p_y^2}} & 0 & 0 \end{bmatrix}_{\bar{\mathbf{x}}_k}$$

Note

Notice how with the EKF we now denote H with a time index subscript k like the state estimate, Kalman Gain, and other values calculated at each iteration of Motion update step or Measurement update step. With the EKF we will now calculate the Measurement Jacobian H for each step evaluated at our state estimate after our motion update state $\bar{x}$

This H_k (not constant) will now play the same role as H (which is constant) in the linear Standard Kalman Filter.

$$K_k = \bar{P} H_k^T (H_k \bar{P} H_k^T + R)^{-1}$$

$$\hat{x}_k = \bar{x}_k + K_k(z_k - h(\bar{x}_k))$$

$$P_k = (I - K_k H_k)\bar{P}_k$$

We use the same logic for our Motion update state with the Dynamics Jacobian F_k:

$$F_k = \left.\frac{\partial f}{\partial \mathbf{x}}\right|_{x_{\hat{k-1}}}$$

$$\bar{x}_k = f(\hat{x}_{k-1}, u_k)$$

$$\bar{P}_k = F_k \hat{P}_{k-1} F_k^T + Q$$

3 The Jacobian as a Change of Basis

The Jacobian matrix is the linear map that best approximates a nonlinear transformation at a point. This insight is from thinking about matrices as linear transformations (for which there is a beautiful intuition in 3Blue1Brown video by Grant Sanderson).

By the definition of a derivative for a smooth function f near a point x_0:

$$f(x_0 + \delta x) \approx f(x_0) + \mathbf{J}\delta x$$

That is to say, the Jacobian matrix linearly maps small perturbations δx in the input space to perturbations in the output space. We interpret the i-th column of the Jacobian Matrix as the result of applying a transformation to the i-th basis vector of the tangent space of the nonlinear function. So each column of $\mathbf{J}$ answers the question : "if I nudge along a basis direction, where does the output go?"

Often in working with robotic you may find yourself converting between joint space and task space representations. Here you will explicitly see the Jacobian treated as a change of basis. Here is an example of swapping between joint space velocities and task space velocities.

$$\dot{x} = \mathbf{J}(q)\dot{q}$$

4 EKF Connection to Euler Method for ODE

Euler Method used to approximate the next state of a scalar ODE is:

$$x(t + \Delta t) \approx x(t) + \Delta t \cdot \frac{dx}{dt}$$

EKF estimating a nonlinear function around the current estimate:

$$h(x) \approx h(\bar{x}) + \left.\frac{\partial h}{\partial \mathbf{x}}\right|_{\bar{x}} (x - \bar{x})$$

Because both methods approximate by assuming that for a small enough local region around some point the function behaves linearly both methods have the same failure modes: the estimate becomes less accurate for larger step sizes (for ODEs times steps and for EKF when the true state is far from the predicted state $\bar{x}$). The EKF will diverge for highly nonlinear systems (for some notion of nonlinearity) and poor initial estimates.

5 Additional Resources

5.1 Jacobian

- Change of Variables and the Jacobian (YouTube video) `https://youtu.be/hhFzJvaY__U?si=5HAcnACCmhKLpvCH`
- Modern Robotics by Kevin Lynch and Frank Park Chapter 5: Velocity Kinematics and Statics (Textbook) `https://modernrobotics.northwestern.edu/chapters/chapter5/`

5.2 Kalman Filters

- (Short online blog) Bzarg Kalman Filters with great visuals `https://www.bzarg.com/p/how-a-kalman-filter-works-in-pictures/`
- KalmanFilter.net (Website, single page overview, and textbook) `https://kalmanfilter.net/`
- Extended Kalman Filter tutorial for non experts`https://simondlevy.github.io/ekf-tutorial/`
- Notational Derivation of Kalman Filters (RoboJackets Software Training) `https://youtube.com/playlist?list=PL1R5gSylLha0j_tmn3YhTTs90-pUFuZH9`